

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Exercício: algoritmos iterativos: especificação, corretude e análise](#)

Exercício: algoritmos iterativos: especificação, corretude e análise?

Fase de avaliação

Fase de configuração	Fase de envio	Fase de avaliação	Fase de cálculo da nota da avaliação	Encerrado
		Fase atual ●		
	Envie seu trabalho Prazo limite dos envios: quinta, 22 set 2022, 23:59 (1394 dias atrás)	Prazo limite da avaliação: quinta, 29 set 2022, 23:59 (1387 dias atrás)		

Seu envio ▼

Você não enviou seu trabalho ainda

Instruções para avaliação ▼

Respostas

1. a. Medida de progresso: podemos usar $j - i$, pois em cada iteração aumentamos o i , ou reduzimos o j , ou ambos. Ou seja, em cada iteração a expressão $j - i$ reduz em pelo menos 1. O algoritmo termina quando atinge $j - i < 0$.
- b. No código pré-loop fazemos i receber o valor 2 e j receber o valor n . Como o único elemento em posição menor que i é o pivô (na posição 1), então INV1 é satisfeita, pois o pivô tem valor menor ou igual ao pivô. Como j está na última posição, nenhum elemento está em posição maior que j , satisfazendo assim INV2 por vacuidade (nenhum elemento em posição maior que j fere a propriedade de ser maior que o pivô).
- c. Os passos do código do loop são apenas os informados nos passos básicos (enunciado):
Se $L[i] \leq \text{pivo}$, avance o i .
Caso contrário, se $L[j] > \text{pivo}$, avance o j .
Caso contrário, troque $L[i]$ com $L[j]$ e avance i e j .
O invariante do loop é mantido pelo código do loop:
Podemos assumir que INV1 e INV2 são verdadeiros no início da iteração.
Caso I: $L[i] \leq \text{pivo}$.
Neste caso apenas avançamos i para $i' = i + 1$, incluindo assim o elemento $L[i]$ no conjunto dos que possuem posição menor que i' . Como $L[i] \leq \text{pivo}$, o INV1 não foi violado por esta ação. Além disso, o INV2 não foi afetado.
Caso II: $L[j] > \text{pivo}$.
Neste caso apenas avançamos j para $j' = j - 1$, incluindo assim o elemento $L[j]$ no conjunto dos que possuem posição maior que j' . Como $L[j] > \text{pivo}$, o INV2 não foi violado por esta ação. Além disso, o INV1 não foi afetado.
Caso III: $L[i] > \text{pivo}$ e $L[j] \leq \text{pivo}$.
Neste caso trocamos $L[i]$ com $L[j]$, e avançamos i para $i' = i + 1$ e j para $j' = j - 1$. Os avanços de i e j fazem $L[i]$ e $L[j]$ pertencerem aos conjuntos dos elementos com posição menor que i' e maior que j' , respectivamente. Porém, após a troca de $L[i]$ com $L[j]$ ficamos com $L[i] \leq \text{pivo}$ e $L[j] > \text{pivo}$, de modo que INV1 e INV2 não são violados pelas ações.
- d. A condição de saída diz que $j < i$, e o invariante do loop diz que os elementos à esquerda de i são menores ou iguais ao pivô, e os elementos à direita de j são maiores que o pivô. Por contradição assumamos que $j < i - 1$, o que implica que existe um inteiro w tal que $j < w < i$. Neste caso, pelo invariante do loop temos $L[w]$ é ao mesmo tempo menor ou igual ao pivô e também maior que o pivô (contradição).

e. No código pós-loop resta apenas colocar o pivô na posição k , onde k é o número de elementos com valor menor ou igual ao pivô. Esta posição k pode ser facilmente obtida pelo valor do índice j ao final do loop, pois como $j = i - 1$ (veja item anterior), e pelo invariante do loop, temos que todos os elementos à direita da posição j são maiores que o pivô, e todos os elementos até a posição j são menores ou iguais ao pivô. Desta forma satisfazemos o que é estabelecido nas pós-condições.

f.

```

Algoritmo: Partição(L[1..n])
pre-cond: Lista L[1..n] de números, com n >= 1.
pos-cond: Reposicionamento dos elementos de L conforme indicado no enunciado.

pivo,i,j = L[1],2,n
loop
    //inv-loop: definido no enunciado.
    saia quando j < i
    if L[i] <= pivo
        i = i + 1
    else if L[j] > pivo
        j = j - 1
    else
        troque L[i] com L[j]
        i = i + 1
        j = j - 1
troque L[1] com L[j]

```

g. (i) Quando a lista tem apenas um elemento ($n = 1$), o índice j é inicializado com 1 no código pré-loop. Desta forma o loop vai encerrar no início da 1ª iteração, pois i é inicializado com valor 2. No código pós-loop vamos trocar o elemento da posição 1 com ele próprio. Como o resultado final é válido, concluímos que o algoritmo já funciona para o caso de entrada de tamanho 1.

(ii) Quando nenhum elemento é maior que o pivô, o índice i vai avançar até a posição $n + 1$ e sair do laço, e j vai continuar valendo n . Desta forma vamos trocar o 1º elemento com o último no código pós-loop, produzindo assim um resultado que atende as pós-condições. Portanto nenhuma adaptação é necessária.

(iii) Quando todos os elementos são maiores que o pivô, exceto o próprio pivô, o índice j vai decrementar até a posição 1 e sair do laço, e o índice i vai permanecer na posição 1. Então no código pós-loop vamos apenas trocar o elemento da 1ª posição com ele próprio. Como o resultado final atende as pós-condições, nenhuma adaptação é necessária.

h. No pior caso vamos precisar mover o índice i até o final da lista, ou o índice j até o início da lista, avançando apenas uma posição em cada iteração. Concluímos que o número de iterações é $\Theta(n)$, e cada iteração executa em $\Theta(1)$. Além disso, o código pré-loop e código pós-loop executam em $\Theta(1)$. Portanto, o algoritmo é $\Theta(n) \cdot \Theta(1) + \Theta(1) = \Theta(n)$.

2. algoritmo(n):

1	for i de 1 até n	$O(n)$
2	f(i)	[1]
3	for j de i+1 até n	[2]
4	g(j)	[3]
5	for i de 1 até n	$O(n)$
6	h(i)	$O(n) \cdot O(n^2) = O(n^3)$

[1]

$$\sum_{i=1}^n \Theta(i) = \Theta\left(\sum_{i=1}^n i\right) = \Theta(n^2).$$

[2]

$$\sum_{i=1}^n \sum_{j=i+1}^n \Theta(1) = \Theta\left(\sum_{i=1}^n \sum_{j=i+1}^n 1\right) = \Theta\left(\sum_{i=1}^n (n-i)\right) = \Theta\left(\sum_{k=1}^{n-1} k\right) = \Theta((n-1)(n-1)) = \Theta(n^2).$$

[3]

$$\Theta\left(\sum_{i=1}^n \sum_{j=i+1}^n \log n\right) = \Theta\left(\log n \cdot \underbrace{\left(\sum_{i=1}^n \sum_{j=i+1}^n 1\right)}_{\text{resolvido em [2]}}\right) = \Theta(n^2 \log n).$$

Concluímos que o algoritmo é $2 \cdot \Theta(n) + 2 \cdot \Theta(n^2) + \Theta(n^2 \log n) + \Theta(n^3) = \Theta(n^3)$.

Envios atribuídos para avaliação ▼

Você não recebeu nenhum envio para avaliar



◀ Vídeo: aproximação de somatórios (18min)

Seguir para...

Slides: projeto e corretude de algoritmos iterativos ►

©2020 – Universidade Federal do Ceará – Campus Quixadá.

Todos os direitos reservados.

Av. José de Freitas Queiroz, 5003

Cedro – Quixadá – Ceará CEP: 63902-580

Secretaria do Campus: (88) 3411-9422

 Baixar o aplicativo móvel.